

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5 Total Marks:50 Annexure A

Design Task

Code of Conduct:

I AGNEESHWARAN B / 192524508 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: AGNEESHWARAN B

Annexure A

Design Task

- 1. Hybrid AI Recommendation Engine Design** Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.
- 2. Smart Disaster Detection System** Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework** Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.
4. **Autonomous Supply Chain Optimization System** Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.
5. **Personalized Learning Analytics Platform** Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect

System Architecture Design	Complete, scalable, well structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed

Data Flow & Processing Pipeline	Clear, logical, and well defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well organized report	Mostly clear	Somewhat organized	Poor presentation

ANSWER SHEET – DESIGN TASK

Course: Artificial Intelligence (CSA17)

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures. **1. HYBRID AI RECOMMENDATION ENGINE DESIGN**

Objective:

The proposed system recommends products, movies, courses or other items by combining content-based filtering and collaborative filtering. PySpark RDDs provide distributed processing for large-scale user-item interaction data, while intelligent agents adapt recommendations using real-time behavior and feedback.

System Architecture:

User Data + Item Data → Data Ingestion → PySpark RDD Processing → Feature Extraction
→ Content-Based Model + Collaborative Filtering Model → Hybrid Recommendation Engine
→ Intelligent Recommendation Agent → User → Feedback → Continuous Update

PySpark and RDD Operations:

- `parallelize()` can create RDDs from local collections.
- `textFile()` can load large datasets.
- `map()` transforms records into useful key-value structures.
- `filter()` removes invalid or irrelevant records.
- `reduceByKey()` aggregates ratings, clicks or interactions.
- `join()` combines user, item and rating information.
- `groupByKey()` can group interactions, although `reduceByKey()` is generally preferred for efficient aggregation.
- `cache()` keeps frequently reused RDDs in memory.
- `collect()` can retrieve small final results to the driver.

Content-based filtering recommends items similar to those previously liked by the user using item features.

Collaborative filtering uses similarities or rating patterns among users/items. A hybrid score can combine both outputs.

Intelligent Agent:

The recommendation agent observes searches, clicks, ratings, purchases and time spent. It selects recommendations, receives feedback and updates the recommendation strategy. Positive feedback increases the preference score, while negative feedback decreases it.

Scalability:

RDDs divide large datasets across cluster nodes so transformations can run in parallel. Partitioning, caching and distributed execution allow the system to support millions of interactions. Fault tolerance is provided because RDDs can be reconstructed from their lineage.

2. SMART DISASTER DETECTION SYSTEM

Objective:

The system detects floods, earthquakes and similar disasters by combining satellite images, sensor readings and social media information. It provides early warnings and helps coordinate response actions.

System Architecture:

Satellite Images —┐
Sensors —————┴→ Data Ingestion → PySpark RDD Processing → Data Cleaning
Social Media ———┐ → Feature Extraction → Multi-source Data Fusion
→ Disaster Detection Model → Intelligent Agents → Alert & Response System

RDD Processing:

- `map()` extracts useful fields and features.
- `filter()` removes noisy or invalid observations.
- `flatMap()` extracts multiple events or keywords from social-media records.
- `reduceByKey()` aggregates sensor readings by location/time.
- `join()` combines information from different sources.
- `cache()` supports repeated analysis.

Data Fusion:

Sensor data provides numerical measurements such as water level, vibration and temperature. Satellite data provides geographical and visual information. Social media can provide rapid reports from affected locations. The sources can be aligned using location and time and combined into a unified risk score.

Intelligent Agents:

A monitoring agent observes incoming data. A detection agent estimates disaster probability. A verification agent compares evidence from different sources. A warning agent generates alerts when the risk exceeds a defined threshold. A response agent recommends actions such as evacuation, rescue-team allocation and resource movement.

Scalability:

PySpark distributes processing across nodes, allowing large satellite datasets, sensor streams and social-media records to be processed in parallel. The architecture should include fault tolerance, data partitioning and monitoring to support continuous operation.

3. AI-BASED FRAUD DETECTION FRAMEWORK

Objective:

The framework identifies suspicious financial transactions from large-scale banking or payment data using distributed PySpark processing, anomaly detection and collaborating intelligent agents.

System Architecture:

Transaction Data → Data Ingestion → PySpark RDDs → Cleaning & Feature Engineering
→ Distributed Anomaly Detection → Risk Scoring → Intelligent Fraud Agents
→ Alert Generation → Investigation → Feedback → Model Improvement

RDD Operations:

- `map()` converts raw transactions into structured records.
- `filter()` removes invalid transactions.
- `map()` can calculate features such as transaction amount, frequency and location difference.
- `reduceByKey()` calculates customer-level statistics.
- `join()` combines transaction records with customer or account information.
- `cache()` stores frequently reused data.

Anomaly Detection:

The system can identify unusual transaction amounts, abnormal transaction frequency, unusual locations, repeated failed transactions or behavior different from a customer's normal pattern. A risk score can combine these signals.

Intelligent Agents:

1. Transaction Monitoring Agent – observes transactions.
2. Anomaly Detection Agent – identifies unusual patterns.
3. Risk Assessment Agent – assigns a fraud-risk score.
4. Alert Agent – triggers alerts for high-risk transactions.
5. Learning Agent – uses investigator feedback to improve future detection.

Distributed Design:

Transactions are partitioned across worker nodes. Parallel processing reduces the time needed to analyse large transaction volumes. RDD lineage provides fault recovery. The system should also use secure access control, encryption and privacy protection for financial data.

4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM

Objective:

The system optimizes inventory, logistics and demand forecasting using PySpark and multiple intelligent agents. The main goal is to minimize total cost while maintaining product availability and delivery efficiency.

System Architecture:

Sales + Inventory + Supplier + Transport Data
→ Data Ingestion → PySpark RDD Processing → Demand Forecasting
→ Inventory Optimization + Route Analysis → Multi-Agent Decision System
→ Supplier/Inventory/Logistics Actions → Monitoring & Feedback

RDD Transformations:

- `map()` transforms sales and inventory records.
- `filter()` removes invalid records.
- `reduceByKey()` aggregates demand by product and location.
- `groupByKey()` groups information when required.
- `join()` combines demand with inventory and supplier information.
- `sortByKey()` can help rank products or routes.
- `cache()` stores frequently used datasets.

Intelligent Agents:

- Inventory Agent – monitors stock levels and recommends replenishment.
- Demand Agent – analyses demand patterns and forecasts future requirements.
- Logistics Agent – selects efficient transportation and delivery options.
- Supplier Agent – monitors supplier availability, lead time and cost.
- Coordination Agent – combines agent decisions and selects a feasible global plan.

Decision Making:

The system can use objectives such as minimizing inventory holding cost, shortage cost, transportation cost and delay. Agents communicate current states and recommendations before selecting actions.

Scalability:

Large transaction, inventory and logistics datasets are partitioned across worker nodes. Parallel RDD operations support rapid processing, while fault tolerance helps the system continue after worker failures. The system can be extended by adding nodes as data volume increases.

5. PERSONALIZED LEARNING ANALYTICS PLATFORM

Objective:

The platform analyses student learning behavior and performance using PySpark RDDs and intelligent agents. It provides adaptive content and real-time feedback for thousands of learners.

System Architecture:

Student Activity + Marks + Attendance + Quiz Data

→ Data Ingestion → PySpark RDD Processing → Feature Extraction

→ Learner Performance Analysis → Student Profile

→ Intelligent Adaptive Learning Agent → Personalized Content

→ Student Feedback → Continuous Learning

RDD Operations:

- map() creates structured student activity records.
- filter() removes incomplete or invalid records.
- reduceByKey() calculates average marks, activity counts or time spent.
- join() combines student profiles with assessment and activity data.
- groupByKey() can organize learning events by student.
- cache() improves performance when learner data is repeatedly accessed.

Learning Analytics:

The system can analyse marks, quiz performance, attendance, time spent, attempted questions and learning progress. Students can be classified into performance levels such as low, medium and high based on defined criteria.

Intelligent Agents:

1. Monitoring Agent – tracks student activity.
2. Assessment Agent – analyses quiz and assignment performance.
3. Recommendation Agent – selects suitable learning materials.
4. Feedback Agent – gives immediate guidance.
5. Adaptation Agent – changes content difficulty according to learner progress.

Adaptive Learning:

If a learner performs poorly on a topic, the agent can recommend simpler explanations, examples and additional practice. If performance improves, the system can provide more advanced content. Feedback is continuously used to update the learner profile.

Scalability:

PySpark distributes analytics for thousands of learners across cluster nodes. RDD transformations allow large volumes of activity data to be processed in parallel. The platform can therefore support scalable dashboards, personalized recommendations and near-real-time learning analytics.